

Project Request: CloudMentor AI – AWS Documentation RAG Assistant

Build a production-quality Retrieval-Augmented Generation (RAG) application called **CloudMentor AI**.

Project Objective

Create an AI-powered AWS Documentation Assistant that allows users to ask questions about AWS services and receive accurate answers based on AWS documentation.

The system must use a RAG architecture:

User Question → Retriever → Vector Database → Relevant AWS Documentation Chunks → LLM → Final Answer with Citations

Example Questions:

- What is Amazon VPC?
- Explain AWS Lambda.
- Difference between EC2 and Lambda.
- How does S3 versioning work?
- What are the pricing models of EC2?

The system must always answer from retrieved documentation and display document citations.

Technical Stack

Frontend:

- Streamlit

Backend:

- Python 3.11+

RAG Framework:

- LangChain

Embedding Model:

- sentence-transformers/all-MiniLM-L6-v2

Vector Database:

- ChromaDB

LLM:

- Google Gemini (gemini-1.5-flash or latest stable model)

Document Processing:

- PyPDF

Deployment Ready:

- Docker
- AWS EC2

Version Control:

- Git

Required Project Structure

cloudmentor-ai/

├── app.py

├── config.py

├── requirements.txt

├── .env.example

├── README.md

├── docs/

| ├── ec2.pdf

| ├── lambda.pdf

| ├── s3.pdf

| └── vpc.pdf

```
|—— data/

|—— vector_db/

|—— src/

| |—— document_loader.py

| |—— text_splitter.py

| |—— embeddings.py

| |—— vector_store.py

| |—— retriever.py

| |—— rag_chain.py

| |—— citation_handler.py

| |—— chat_memory.py

| |—— utils.py

|—— tests/

|—— Dockerfile
```

Functional Requirements

1. PDF Document Ingestion

Create a document ingestion pipeline that:

- Reads all PDF files from docs/
- Extracts text
- Preserves metadata
- Stores source filename
- Stores page number

Metadata must include:

```
{ "source": "ec2.pdf", "page": 10 }
```

2. Document Chunking

Implement RecursiveCharacterTextSplitter.

Settings:

chunk_size = 1000

chunk_overlap = 200

Requirements:

- Preserve context
- Keep metadata attached
- Generate clean chunks

3. Embeddings Layer

Use:

sentence-transformers/all-MiniLM-L6-v2

Requirements:

- Generate embeddings
- Cache embeddings when possible
- Support future model replacement

4. ChromaDB Vector Database

Store all document embeddings.

Requirements:

- Persistent storage
- Automatic reload on restart
- Efficient semantic search

Persist location:

vector_db/

5. Retriever Layer

Implement semantic search.

Requirements:

- Top K retrieval
- Default K = 5
- Configurable retrieval settings

Return:

- Retrieved chunk text
 - Source filename
 - Page number
-

6. Gemini Integration

Use Google Gemini API.

Requirements:

- Read API key from .env
- Error handling
- Rate limit handling
- Configurable model selection

Never hardcode API keys.

7. RAG Pipeline

Workflow:

1. User enters question
2. Retrieve top relevant chunks
3. Build context
4. Send context + question to Gemini
5. Generate answer
6. Display citations

Prompt template:

"You are CloudMentor AI, an AWS Documentation Assistant.

Answer only from the provided AWS documentation context.

If information is unavailable in the context, clearly state that the answer could not be found in the provided AWS documentation.

Provide concise, accurate explanations."

8. Citation System

Every answer must display sources.

Example:

Answer: Amazon VPC enables users to create isolated virtual networks in AWS.

Sources:

- vpc.pdf (Page 15)
- vpc.pdf (Page 18)

Never return answers without citations.

9. Streamlit User Interface

Create a modern professional UI.

Features:

Main Chat Interface

- ChatGPT-style layout
- User messages
- Assistant messages

Sidebar

- Application title
- Loaded document count
- Vector database status
- Clear chat button

Chat Features

- Persistent chat history

- Follow-up questions
 - Session management
-

10. Upload New PDFs

Allow users to upload AWS PDF documents.

Requirements:

- Upload through UI
- Automatically process
- Generate embeddings
- Update vector database

No manual indexing required.

11. Chat Memory

Implement conversation memory.

Requirements:

- Store previous messages
- Enable contextual follow-up questions
- Session-based memory

Example:

User: What is EC2?

User: How is it different from Lambda?

The system should understand "it" refers to EC2.

12. AWS Service Comparison Mode

Add support for comparison questions.

Example:

Compare EC2 and Lambda.

Output:

Feature	EC2	Lambda
---------	-----	--------

Generate structured comparisons when possible.

13. Architecture Recommendation Feature

When users ask:

"Design architecture for an ecommerce website"

Provide:

- Recommended AWS services
- Architecture explanation
- Suggested AWS components

Only use AWS services documented in available context.

14. Error Handling

Handle:

- Missing PDFs
- Empty vector database
- Invalid API key
- API timeout
- No search results

Display user-friendly messages.

15. Logging

Implement logging.

Log:

- Questions
- Retrieval results
- API errors
- System events

Store logs locally.

16. Security

Requirements:

- Environment variables for secrets
 - Input validation
 - Secure file uploads
 - File size limits
-

17. Testing

Create tests for:

- Document loading
- Chunking
- Embeddings
- Retrieval
- RAG responses

Use pytest.

18. Docker Support

Create Dockerfile.

Requirements:

- One-command deployment
 - Production-ready configuration
-

19. Documentation

Generate a professional README including:

- Project overview
- Architecture diagram
- Installation steps
- Environment setup
- Usage guide

- Deployment instructions
 - Screenshots placeholders
-

20. Code Quality

Requirements:

- Modular architecture
 - Clean code
 - Type hints
 - Comments where necessary
 - PEP8 compliance
 - Production-quality structure
-

Expected Deliverables

Generate:

1. Complete source code
2. Folder structure
3. requirements.txt
4. Dockerfile
5. README.md
6. .env.example
7. Streamlit UI
8. ChromaDB integration
9. LangChain integration
10. Gemini integration
11. Citation system
12. Upload functionality
13. Chat memory
14. Architecture recommendation module
15. Test suite

The application should be runnable using:

```
pip install -r requirements.txt
```

```
streamlit run app.py
```

and should function as a complete AWS Documentation RAG Assistant ready for GitHub portfolio deployment.